Deployment Compatibility Contract

What DeployGuard can deploy automatically, what evidence it requires, and when it must block instead of guess.

> **Core promise**
> If every selected service fits a supported v1 deployment contract, DeployGuard owns deterministic repository/build/runtime handling. If the repository falls outside the contract, deployment must be blocked before GitHub Actions or AWS.

Supported v1 contracts

Contract	Supported structure	Typical examples
JS_STANDALONE	One JavaScript/TypeScript app at the selected service root	Next.js, NestJS, Express, React/Vite, Node API
JS_WORKSPACE_MEMBER	Selected package inside a declared npm/pnpm/Yarn/Bun workspace	Next + Nest monorepo, React + Node workspace, shared packages
PYTHON_STANDALONE	One recognized Python app/package within the selected service boundary	Django, Flask, FastAPI
MULTI_SERVICE	1-20 selected services; every service must satisfy a supported contract	React + FastAPI, Next + Nest, frontend + API

> **Framework names are examples, not contracts.**
> DeployGuard should classify repository structure and deployment ownership, not build separate logic for every framework.

Managed database combinations

Supported: no managed database, PostgreSQL, MySQL, or MongoDB. In v1, at most one managed database is attached to one application service.

CONTRACT A

JavaScript / TypeScript standalone

A selected service is deployable as a standalone JS/TS service when it has its own package manifest and is not owned by a declared workspace.

```
my-api/
├── package.json
├── package-lock.json  (or pnpm-lock.yaml / yarn.lock / bun.lock)
├── src/
│   └── main.ts
└── ...
```

DeployGuard fact	Expected value
serviceDirectory	my-api
workspace owner	none
buildRoot / installRoot	my-api
execution target	my-api
build engine	Railpack

Examples that fit this contract

- NestJS API with package.json and its own build/start scripts.
- Express or Fastify API with package.json.
- Next.js application at the selected root.
- React/Vite frontend at the selected root.
- Any other JS/TS application whose package manager and runtime contract are deterministic.

What DeployGuard must resolve

- Exact source SHA and selected service directory.
- Canonical package/build scope.
- One canonical application port.
- Required build/runtime configuration.
- Managed database attachment, if selected.
- Railpack image build followed by runtime validation.

> **Application source errors remain application errors.**
>
> Matching the deployment contract does not guarantee that broken code compiles or starts. DeployGuard guarantees correct platform handling of the supported contract.

CONTRACT B

JavaScript / TypeScript declared workspace

A workspace service is deployable when the selected package is deterministically a member of a declared npm, pnpm, Yarn, or Bun workspace.

```
repo/
├── pnpm-workspace.yaml
├── package.json
├── pnpm-lock.yaml
└── packages/
    ├── client/
    │   └── package.json
    ├── server/
    │   └── package.json
    └── shared/
        └── package.json
```

Selected service	Workspace context	Execution target
packages/client	repo root	client package only
packages/server	repo root	server package only
packages/shared	Not selected	library/dependency - not deployed automatically

Contract rules

- Declared workspace ownership is authoritative build/install context.
- The selected package remains the deployment target.
- Shared packages remain available to the build but are not automatically promoted to services.
- Nested workspaces use the nearest deterministic owning workspace.
- Ambiguous or unresolvable workspace ownership must be blocked, not downgraded to standalone.

> **Important distinction**
>
> Workspace root = build context. Selected package = deployable application. DeployGuard must preserve both facts.

Typical deployable combinations

Repository	Deployable services	Shared code
Next.js + NestJS pnpm workspace	web + api	shared package
React + Express npm workspace	frontend + backend	types/utilities
Multiple Node APIs in Yarn workspace	api-a + api-b + worker*	common libraries

*Only services that satisfy the runtime/port contract are deployable as web services.

CONTRACT C

Python standalone and mixed multi-service

A Python service is deployable when its selected service directory contains a recognized Python package/application contract and its required build inputs stay inside the supported service boundary.

```
api/
├── pyproject.toml        (or requirements.txt / Pipfile)
├── app/
│   └── main.py
└── ...
```

Recognized Python packaging

Packaging evidence	Typical application examples
requirements.txt / pip	Django, Flask, FastAPI
pyproject.toml	Modern Python applications
Poetry	Poetry-managed web applications
uv / PDM / Pipenv	Supported standalone package layouts

V1 boundary

Repository-local Python path dependencies that require files outside the selected service boundary should be blocked unless DeployGuard has an explicitly supported common-root contract.

Multi-service combinations

A multi-service repository is supported only when every selected service independently passes a supported contract.

Combination	Example	Managed DB example
JS + JS	React frontend + NestJS API	PostgreSQL attached to API
JS + Python	Next.js frontend + FastAPI API	PostgreSQL attached to API
JS workspace	Next.js + NestJS workspace	MySQL attached to server
Python + JS	Django API + Vite frontend	MySQL attached to Django
Single API	Express API	MongoDB attached to API
Single Python API	FastAPI	PostgreSQL attached to API

One bad service blocks the deployment.

If any selected service is unsupported, ambiguous, or invalid, the whole multi-service deployment must stop before dispatch.

ADMISSION & BLOCKING

Configuration contract and fail-closed behavior

Configuration admission

State	Meaning	Action
READY	All required platform/repository configuration is satisfied	Deployment may continue
INPUT_REQUIRED	Required user configuration is missing	Ask user; do not dispatch
BLOCKED	Contract conflict, ambiguity, prohibited override, or unsupported topology	Stop before GitHub/AWS

Platform-owned runtime facts

- PORT is canonical and platform-managed.
- HOST is platform-managed as 0.0.0.0.
- Managed database aliases are platform-owned when a DeployGuard managed database is attached.
- User configuration remains service-scoped.

Repositories DeployGuard must block in v1

- Unknown ecosystem or no supported JS/TS/Python application manifest.
- Languages outside the published v1 contract, even if Railpack itself can build them.
- Workspace evidence exists but selected-package ownership cannot be resolved deterministically.
- Duplicate/ambiguous package identities or missing workspace dependencies.
- JS local sibling dependency outside a supported workspace boundary.
- Python sibling/path dependency outside the supported service boundary.
- Unresolved, conflicting, or invalid application port.
- Repository/build path that escapes the repository.
- Any multi-service project containing one unsupported selected service.

> **Public contract statement**
>
> Inside the supported contract -> DeployGuard proceeds. Outside the supported contract -> DeployGuard blocks before cloud execution. DeployGuard does not guess unknown repository topology.

Recommended user-facing wording

DeployGuard v1 supports standalone JavaScript/TypeScript applications, declared JavaScript/TypeScript workspace members, standalone Python applications, and multi-service combinations of those contracts. Managed PostgreSQL, MySQL, and MongoDB are supported. Repositories that cannot be deterministically classified into a supported contract are blocked before deployment and require a supported structure or a future explicit compatibility path.